

Designing a Large-Scale Web Crawler: Notes on Architecture

These notes work through the design of a web-scale crawler from first principles, with particular attention to two things that are easy to get wrong: the shape of the single-machine scheduler, and the choice of sharding key in the distributed case. Sources are marked throughout — published literature, black-box systems, and practical experience are kept distinct.

1. The Single-Machine Scheduler: Event-Driven, Not Lookup

1.1 The trap of the pull model

Suppose you specify the scheduler as a synchronous query:

```
next_ready(t) -> url | None
```

That is: the caller supplies a timestamp `t` and asks which URL should be dispatched at that instant. This interface manufactures a problem out of nothing:

- The primary ordering is by earliest arrival.
- Eligibility depends on `host_last_sent + delay <= t`.

These are two independent dimensions, and they cannot be collapsed into a single ordered structure. Forcing it requires two heaps (one keyed on arrival, one on ready time), state migration between them, lazy deletion, and an additional assumption that `t` is monotonically non-decreasing — without which the migration is irreversible and the whole thing degrades to a full scan.

Every one of those difficulties is an artifact of the interface, not something intrinsic to crawl scheduling.

1.2 The event-driven shape

A real crawler is two multiplexed event sources. Events are pushed to you; nothing needs to be searched for.

- **Which of the `K` sockets is ready** — handled by `epoll/select` (the kernel).
- **Which of the `N` hosts has come off cooldown** — handled by a timer (delay queue or timing wheel).

Single-machine state:

Structure	Contents	Purpose
<code>m1</code>	<code>host -> {URL queue, delay, last_sent}</code>	Source of truth per host
<code>timer</code>	Min-heap keyed on <code>ready_time</code>	Who thaws when
<code>ready deque</code>	Hosts that have gone green, awaiting dispatch	FIFO, take from the head
Connection pool	K connection slots	Concurrency ceiling

Three event entry points:

1. **add** — new URL enters `m1`; if the host is idle, it goes straight to the ready deque.
2. **timer fires** — a host goes green; take its queue head from `m1` and push to the ready deque.
3. **connection frees up** — take one from the ready deque and dispatch; re-arm the timer for that host at `t + delay`.

1.3 Three things that disappear

- **The explicit `inflight < K` check.** K becomes the size of a resource pool. With no free slot there is no dispatch action, so exceeding the limit is structurally impossible rather than being prevented by a condition you might write wrong.
- **The `complete()` interface.** Politeness depends only on the moment of dispatch — `ready_time = dispatch_time + delay` — and not on when the response comes back. Releasing a concurrency slot is an internal I/O callback, not an external API.
- **The need to sort the ready set.** Hosts go green in time order by construction, since the timer is keyed on expiry. So the ready set can be a deque rather than a heap: take the head in O(1). Making it double-ended lets retries and high-priority URLs jump in at the front.

The reason the pull model needs an arrival-ordered heap is that all hosts appear eligible simultaneously, so their relative ordering is lost and must be reconstructed from arrival times. The push model preserves that ordering in the timer.

2. The Frontier: What to Fetch, in What Order

2.1 Not a DFS

Seed list → fetch → extract links → new URLs into the frontier → iterate. The shape resembles BFS, but what actually determines order is priority, not traversal strategy.

2.2 Priority keys

The keys that matter in practice:

- PageRank or site quality
- Time of last fetch (freshness)
- Estimated update frequency
- Historical status: 404s, 5xx, robots exclusions

Global FIFO on arrival time has no practical value in web crawling. When a tie-break rule like "prefer the earliest arrival" shows up in an interview question, it is usually there to make the problem decidable, not because it reflects production practice.

2.3 Mercator's two-tier queue

- **Front queues** manage priority.
- **Back queues** manage politeness.

Priority and politeness are orthogonal concerns and get handled in separate layers.

3. Distribution: Choosing the Sharding Key

3.1 Shard by host, not by URL

This is a textbook conclusion with clear published support:

- **Manning's IR textbook** poses "why is it better to partition hosts rather than individual URLs" as an exercise — it treats the answer as established.
- **Mercator / Atrax**: in the distributed configuration the hostname space is partitioned among processes, each responsible only for its subset. Later versions partition the URL space by hostname and avoid the bottleneck of a centralized URL server.
- **UbiCrawler**: consistent hashing on the host component of the URL.
- **Compaq's distributed crawler patent**: assigning URLs by hostname makes politeness policies more practical to enforce and avoids replicating per-host information across multiple crawlers.
- **BUBiNG**: the UbiCrawler authors' successor system, fully distributed, published.

3.2 Why URL hashing fails

Hashing on the URL scatters a single host's URLs across every machine. Consequences:

1. **Politeness breaks.** Each of 100 machines believes it is respecting a 10 QPS limit; collectively they emit 1000. A distributed quota scheme becomes necessary as a patch.

2. **Per-host state has nowhere to live.** robots.txt (and its ~24h cache), DNS resolution results, crawl-delay, the host's recent response history. Either every machine keeps a copy — wasteful and inconsistent — or every check goes cross-machine. This holds even with a perfect central rate limiter; it is independent of the rate-limiting problem.
3. **Cache locality is lost.** Manning explicitly names cache locality as the key goal of a distributed crawl architecture.
4. **Statically split quotas sit idle.** A host's URLs typically arrive in bursts: one round of link extraction yields 5000, then nothing for a long time. Split evenly across 100 machines, most quota goes unused while the machine that actually has work is capped at $1/N$. Throughput ends up far below what the host would permit. Fixing it means letting quota flow, and flowing quota needs central coordination — precisely what Mercator set out to avoid.

3.3 What host sharding costs

- **Load skew.** A handful of very large hosts may hold a million times the URLs of a small one.
- **Rehashing.** Adding or removing machines forces migration, during which politeness is briefly unenforceable. Consistent hashing mitigates this but does not remove it.
- **Hostname vs IP granularity.** The Compaq patent makes a subtle point: if you distribute by symbolic hostname, two distinct hostnames may resolve to the same IP and land on different crawlers, which will then hit the same physical server in parallel. Distributing by resolved IP guarantees a single crawler per host machine. On shared hosting this is a genuine hole.

3.4 Three sharding keys coexist

A real crawler uses at least three different partitioning keys:

Component	Key	Reason
URL frontier	host	Politeness and per-host state locality
URL dedup (Bloom filter)	URL hash	Needs no per-host state
Content dedup (fingerprints/shingles)	fingerprint property	Identical content appears on different hosts — cannot be host-partitioned

The last one is explicit in Manning: document fingerprints cannot be partitioned by hostname, because nothing prevents the same or near-identical content from appearing on different web servers.

4. Scheduling Ownership vs Physical Fetching

An important clarification. "One host belongs to one machine" strictly means **the scheduling decision is made at a single decision point** — not necessarily that the same machine executes the fetch.

The decoupled form:

```
frontier / scheduler    (ownership by host)
    ↓
host-level politeness decision
    ↓
dispatch fetch task
    ↓
many fetcher machines
```

This is necessary under specific conditions:

- Fetching must be geographically distributed (egress from different regions).
- Fetching is resource-heavy (JS rendering in a browser).
- The fetcher tier has its own scaling requirements.

But it is not the norm. Every large-scale system in the published literature — Mercator, UbiCrawler, Manning's host splitter — binds hosts to physical nodes. Decoupling means every URL passes through a central decision point, reintroducing the bottleneck Mercator was designed to avoid (batched tokens or leases amortize it, but not for free), and the per-host state problem from § 3.2 remains regardless.

The accurate statement: **host sharding is the default; separating scheduling from fetching is an evolution for when the fetch tier has independent constraints, not a replacement.**

5. Tiering for Very Large Hosts

Don't pay coordination costs for 100% of hosts. Pay them for the 0.01% that need it.

Tier	Scope	Strategy
Long tail	The overwhelming majority	Hash to a single machine; politeness is a purely local decision; zero coordination. One machine handles tens of thousands of small hosts.
Giants	A handful of very large hosts	A dedicated small cluster sharing a host-level quota

The boundary should be dynamic, not a static list. Promote a host to cluster mode when its frontier backlog crosses a threshold or its single-machine quota stays saturated. The distribution of host sizes on the web shifts over time; a static list goes stale.

Quota allocation within the cluster — two approaches, with different applicable conditions:

- **Static even split.** When the target host's URL volume is large and sustained (deep, targeted crawling of a single site), every machine always has work and no quota sits idle. This is sufficient in practice; adding coordination only adds complexity.
- **Leases / batched tokens.** Needed when URLs arrive in bursts. The limiter hands out N units of quota plus a time window; the machine spends them locally and asks again when exhausted. Central QPS drops from once-per-fetch to once-per-N-fetches. The cost is precision at window boundaries, absorbed by setting quotas conservatively.

One trade-off here is not purely technical. The largest hosts tend to also have the strictest robots.txt and the most aggressive IP blocking. Giving them a cluster is partly about throughput and partly about spreading requests across egress IPs — which cuts against the spirit of politeness, depending on whether the other side identifies you by IP or by user agent.

6. Monitoring

6.1 Core signals

- **Per-host actual fetch rate vs allocated quota**
- **Global fetch QPS** — the master health indicator

Alert when either drops below threshold.

6.2 Two causes that must be distinguished

Fetch rate alone cannot localize a problem. Pair it with other indicators:

Observation	Companion signal	Diagnosis
Quota unused, frontier non-empty	—	Real problem: slow DNS, exhausted connection pool, target slowing down
Quota unused, frontier empty	—	Normal — there is simply no work
Fetch rate drops	HTTP status distribution (429 share), response body size distribution	Target-side throttling or degradation, not a fault in your system

The third row is the one that gets misdiagnosed as an internal failure, sending you off to tune the wrong thing.

6.3 A secondary use

Per-host quota utilization is itself an input to **dynamic quota rebalancing**: one machine persistently under-using while another stays saturated indicates misallocation, so move quota. This is simpler than a lease mechanism and is driven by observation rather than per-request coordination.

7. What Is Actually Public

System	Availability
Mercator / Atrax	Published (Najork). The canonical reference.
UbiCrawler / BUBiNG	Published. Consistent-hashing host sharding.
Manning IR textbook, ch. 20	Full architecture, host splitter, the three sharding keys
Compaq distributed crawler patent	The clearest statement of why the sharding key matters, including hostname vs IP
Googlebot	Black box. Only the 1998 Brin & Page paper is public — a centralized URLserver, ~300 open connections per crawler, four machines at roughly 100 pages/second. That architecture is long obsolete. The modern system is known only through behavioral descriptions in Search Central docs and public statements.
Bingbot	Black box. No published system design from Microsoft; only SEO-side behavioral description.

When discussing this, citing Mercator is safer than citing Google: the former has a paper, the latter can only be guessed at. Saying plainly that no architecture documentation exists for Googlebot or Bingbot is better than inventing one.

8. Portable Judgments

1. **The shape of the interface determines the difficulty.** The two-dimensional ordering conflict created by a pull-model `next_ready(t)` does not exist in the push model.

Recognizing that an abstraction has distorted a good problem into a bad one is itself a useful judgment.

2. **Constant factors can beat asymptotics.** Hosts outnumber URLs by several orders of magnitude in the other direction — billions of URLs, millions of hosts, far fewer active at any moment. An $O(H)$ scan over contiguous memory, with good cache behavior, may well outrun an $O(\log H)$ scheme built from two heaps, lazy deletion, and migration — and it comes with an order of magnitude less code, no invariants to maintain, and no assumption that time advances monotonically. Knowing when not to optimize is part of the work.
3. **Spend coordination on the exceptions, not on the default path.**